

Software Test Plan (STP)

Web Based ML Model Evaluator

Project: Web Based ML Model Evaluator

Version: 1.0

Team: Team 5

Authors: Denis Mangla (PES2UG24CS151), R P Pranav (PES2UG24CS917), Chellamani Adithya Iyer (PES2UG24CS129), Disha Bose (PES2UG24CS160)

Course: Software Engineering

Date: 15-09-2026

Status: Final – Ready for Submission

1. Introduction

Purpose. This document defines the test plan for the Web Based ML Model Evaluator v1.0. It states the objectives, scope, strategy, environment, schedule and responsibilities for testing, and establishes how every requirement in SRS v1.1 is traced to the test cases that verify it.

Scope. Testing covers account and access management, dataset upload and validation, the preprocessing pipeline, model selection and configuration, background training execution, metric computation and run comparison, reporting and export, and administration. It also covers the six security requirements and the six non-functional requirements. Excluded are the internals of scikit-learn and pandas, the Django framework itself, and browser rendering engines — all treated as trusted third-party components.

References. SRS v1.1 (source of all MLE-F, MLE-NF and MLE-SR identifiers), Software Architecture and Design Specification v1.0, and the Requirements Traceability Matrix in the SRS appendix.

Definitions. STP — Software Test Plan. SRS — Software Requirements Specification. RTM — Requirements Traceability Matrix. Run — one execution of an algorithm over a prepared dataset. UAT — User Acceptance Testing. IDOR — Insecure Direct Object Reference.

2. Test Items

- AccountsApp — registration, authentication, session lifetime, role separation.
- DatasetsApp — upload, validation, column profiling, dataset ownership.
- PipelineApp — target and feature selection, imputation, encoding, scaling, seeded split.
- ModelsApp — classification and regression algorithm catalogue, hyperparameter validation.
- RunsApp and TrainingWorker — background execution, job state reporting, reproducibility, resource limits.
- EvaluationApp — classification and regression metrics, multi-run comparison.
- ReportsApp — evaluation report generation and CSV export.
- AdminApp — account administration and dataset catalogue management.
- AuditLog — append-only audit records.

3. Features to be Tested

Every feature below is traced to its requirement identifier in SRS v1.1 and to the test-case suite that verifies it.

Requirement	Feature	Component	Test cases
MLE-F-001	Register with unique email and password of at least eight characters	AccountsApp	TC-ACC-01
MLE-F-002	Authenticate before access to any dataset, run or report	AccountsApp	TC-ACC-02
MLE-F-003	Registered User / Administrator role separation	AccountsApp	TC-ACC-03
MLE-F-010	Upload tabular CSV up to 50 MB	DatasetsApp	TC-DS-01
MLE-F-011	Validate extension and structure; reject malformed or undersized files	DatasetsApp	TC-DS-02
MLE-F-012	Profile rows, columns, inferred types and missing-value percentages	DatasetsApp	TC-DS-03
MLE-F-013	Dataset ownership isolation between users	DatasetsApp	TC-DS-04
MLE-F-020	Nominate target column; mark features or exclusions	PipelineApp	TC-PP-01
MLE-F-021	Missing-value strategies: mean, median, mode, row deletion	PipelineApp	TC-PP-02
MLE-F-022	One-hot / label encoding; standardisation / min-max scaling	PipelineApp	TC-PP-03
MLE-F-023	Seeded train/test split between 50:50 and 90:10	PipelineApp	TC-PP-04
MLE-F-030	Five classification algorithms available and trainable	ModelsApp	TC-MS-01
MLE-F-031	Three regression algorithms available and trainable	ModelsApp	TC-MS-02
MLE-F-032	Hyperparameter entry with validated ranges and defaults	ModelsApp	TC-MS-03
MLE-F-040	Background execution; interface not blocked	RunsApp	TC-TR-01
MLE-F-041	Job state Queued / Running / Completed / Failed with reason	RunsApp	TC-TR-02
MLE-F-042	Run reproducible from stored configuration and seed	RunsApp	TC-TR-03
MLE-F-050	Accuracy, precision, recall, F1 and confusion matrix	EvaluationApp	TC-EV-01
MLE-F-051	MAE, MSE, RMSE and coefficient of determination	EvaluationApp	TC-EV-02
MLE-F-052	Side-by-side comparison of two or more completed runs	EvaluationApp	TC-EV-03
MLE-F-060	Downloadable evaluation report with full run detail	ReportsApp	TC-RP-01
MLE-F-061	CSV export matching the on-screen comparison	ReportsApp	TC-RP-02
MLE-F-062	List, deactivate and reactivate accounts	AdminApp	TC-AD-01
MLE-F-063	List and remove dataset catalogue entries	AdminApp	TC-AD-02

Requirement	Feature	Component	Test cases
MLE-NF-001	95th-percentile response time ≤ 2 s at 10 concurrent users	WebUI	TC-PERF-01
MLE-NF-002	Training of $50,000 \times 30$ dataset within 120 s	TrainingWorker	TC-PERF-02
MLE-NF-003	Failed job does not render the application unusable	RunsApp / Worker	TC-REL-01
MLE-NF-004	First-time task completion within 5 min; keyboard reachability	WebUI	TC-UX-01
MLE-NF-005	$\geq 60\%$ unit-test line coverage of the three core modules	Core modules	TC-MNT-01
MLE-NF-006	Run and audit retention with UTC timestamps	Database / AuditLog	TC-OPS-01
MLE-SR-001	TLS 1.2+ enforced; HTTP redirected to HTTPS	Deployment	TC-SEC-01
MLE-SR-002	Passwords stored only as salted one-way hashes	AccountsApp	TC-SEC-02
MLE-SR-003	Server-side role and ownership checks on every request	All components	TC-SEC-03
MLE-SR-004	Upload validation; no pickle or joblib loading	DatasetsApp	TC-SEC-04
MLE-SR-005	10-minute wall-clock limit terminates only the offending job	TrainingWorker	TC-SEC-05
MLE-SR-006	Append-only audit records with actor, action, resource, time	AuditLog	TC-SEC-06

4. Features Not to be Tested

- Internals of scikit-learn and pandas — trusted third-party libraries; our tests verify that we call them correctly and that our reported values match their reference output, not that their algorithms are correct.
- The Django framework itself, including its ORM, session handling and password hasher — we test our configuration and use of them, not their implementation.
- Browser rendering engines and operating-system file-system behaviour.
- Capabilities explicitly out of scope in SRS §1.2: GPU and deep-learning workloads, unsupervised learning, non-tabular datasets, automated feature engineering and model inference endpoints.
- Multi-machine deployment, load balancing and high-availability behaviour — the system is specified to run on a single development machine.

5. Test Approach / Strategy

Levels.

- Unit tests — individual functions in PipelineApp, ModelsApp and EvaluationApp, where correctness is numeric and silent errors are most likely. This is where the coverage target of MLE-NF-005 applies.
- Integration tests — across component boundaries: DatasetsApp to FileStore and Database; RunsApp to TrainingWorker to EvaluationApp.
- System tests — the end-to-end path from upload through preprocessing, training and evaluation to report export.
- Acceptance tests — verification of each requirement against the acceptance criterion stated in the SRS.

Types.

- Functional testing of every requirement listed in section 3.
- Regression testing — the full unit and integration suite is run for every pull request through the automated test pipeline.
- Performance testing — response time under concurrent load, and training duration against the reference dataset.
- Reliability testing — fault injection: a deliberately failing run and a worker terminated mid-run.
- Usability and accessibility testing — a timed first-use walkthrough and a keyboard-only pass.
- Security testing — the manual checklist in section 5.1.

Entry criteria. A build that starts without error, the test dataset fixtures available, the environment provisioned, and the requirement under test implemented and merged to the main branch.

Exit criteria. Every planned test case executed; no open defect of critical or high severity; every high-priority functional requirement passing; no non-functional requirement failing its stated measurement; coverage of the three core modules at or above 60%; and the RTM showing Accepted against every test case.

5.1 Security Validation

- Access control — attempt to read and modify another user’s dataset, run and report by substituting identifiers (IDOR); every attempt must return HTTP 403 (MLE-SR-003, MLE-F-013).
- Privilege escalation — request each administrative page as a Registered User; every attempt must return HTTP 403 (MLE-F-003).
- Upload safety — submit an executable renamed to .csv, a malformed CSV, an oversized file and a crafted pickle payload; each must be rejected, none may be written to FileStore, and no code from them may execute (MLE-SR-004).
- Credential handling — inspect the database and application logs to confirm only salted hashes are stored and no credential material is logged (MLE-SR-002).
- Transport security — confirm the deployed instance answers only over HTTPS and that no protocol below TLS 1.2 is negotiated (MLE-SR-001).
- Resource exhaustion — submit a deliberately long-running job and confirm it is terminated at the limit while concurrent jobs complete normally (MLE-SR-005).
- Audit integrity — perform login, upload, deletion and an administrative action; confirm exactly one audit record each, and that no application route can edit them (MLE-SR-006).

6. Test Environment

Aspect	Detail
Hardware	Standard development laptop, CPU only; no GPU and no special-purpose hardware required.
Software	Python 3.11, Django, pandas, scikit-learn; SQLite in development, PostgreSQL for the deployment check; Chrome, Firefox and Edge at a minimum viewport width of 1024 px.
Tools	pytest with coverage (unit and integration), Jira (defect tracking), Git and GitHub (version control and review), browser developer tools (accessibility and keyboard checks).
Test data	A small well-formed classification CSV and a small regression CSV as golden fixtures with known expected metrics; a 50,000 × 30 reference dataset for performance; and a set of deliberately invalid files — executable renamed to .csv,

Aspect	Detail
	malformed CSV, 19-row file, oversized file and a crafted pickle payload.
Accounts	Two Registered User accounts (to verify isolation between them) and one Administrator account.

7. Test Schedule

Milestones are expressed relative to the start of the test phase, since the course deadlines are set by the coordinator and the plan should stay valid if those dates shift.

Week	Milestone	Owner
Week 1	Test case design for all suites; golden fixtures prepared	Disha Bose
Week 1	Test environment provisioned; CI pipeline running the suite on every pull request	Denis Mangla
Week 2	Unit test execution for PipelineApp, ModelsApp, EvaluationApp; coverage measured	Chellamani Adithya Iyer, Disha Bose
Week 3	Integration and system test execution; defects logged in Jira	All members
Week 3	Performance and reliability test execution	Denis Mangla
Week 4	Security validation checklist execution	Disha Bose
Week 4	Usability and accessibility walkthrough with three participants	R P Pranav
Week 4	Defect fixes, regression run, Test Summary Report	All members

8. Test Deliverables

- This Test Plan.
- Test case specifications for every suite named in section 13.
- Automated test scripts (pytest) held in the repository alongside the code they verify.
- Golden test fixtures and the invalid-file set.
- Test execution logs and automated build/test reports.
- Coverage report for the three core modules.
- Defect reports in Jira, each referencing the requirement it violates.
- Requirements Traceability Matrix updated with execution status.
- Test Summary Report.

9. Roles and Responsibilities

Role	Name	Responsibility
ML Engineer / QA & Test	Disha Bose	Coordinates test-plan and case design; runs security validation and verifies model/metric correctness against reference values.
Data Pipeline Engineer	Chellamani Adithya Iyer	Designs and executes dataset, validation and preprocessing tests, including invalid-file cases; logs defects.

Role	Name	Responsibility
Frontend & Results/Reports	R P Pranav	Supports interface, reporting and export defect reproduction/fixes; leads usability and accessibility walkthroughs.
Team Lead / Backend & Architecture	Denis Mangla	Coordinates test environment and integration/performance/reliability testing; reviews test readiness and sign-off.

10. Risks and Mitigation

Risk	Mitigation
Metrics are wrong in a way no test notices, which is the worst outcome for a tool whose purpose is comparison.	Golden fixtures with independently computed expected values; every metric asserted against scikit-learn reference output within 1e-6 rather than against our own implementation.
Implementation slips and leaves too little time for testing.	Unit tests are written alongside each requirement rather than at the end; the CI pipeline runs the suite on every pull request from Week 1.
The performance test cannot be run because the reference dataset is not ready.	The 50,000 × 30 dataset is generated synthetically as a Week 1 deliverable, so it does not depend on any external source.
Only one member understands a given module, so defects there cannot be triaged if that member is unavailable.	Every change reaches the main branch through a reviewed pull request, so at least two members have read every module.
Flaky timing-dependent tests around the asynchronous training path.	Job state is asserted through the status interface with explicit waits rather than fixed sleeps; the worker is invoked synchronously in unit tests and asynchronously only in integration tests.
Security checklist is treated as a formality and passes vacuously.	Each security test states the specific attack attempted and the expected rejection, so a pass requires the attack to have actually been executed.

11. Assumptions & Dependencies

- The test environment matches the operating environment described in SRS §2.4 and SAD §3.6.
- Golden fixtures and the synthetic reference dataset are available before execution begins.
- The automated test pipeline is configured and running before Week 2.
- Three participants are available for the usability walkthrough in Week 4.
- A deployed instance with TLS is available for the transport security check; where deployment is unavailable, MLE-SR-001 is verified against the deployment configuration instead and the limitation is recorded.
- Requirements are stable at SRS v1.1; any change to a requirement invalidates the test cases traced to it and requires them to be revised.

12. Suspension & Resumption Criteria

Suspend testing if the build fails to start, if a defect blocks more than 30% of the planned test cases, if the test environment is unavailable for more than four hours, or if a requirement under test is changed mid-execution.

Resume testing once the blocking defect is resolved and verified, the environment is stable, and any test cases invalidated by a requirement change have been revised and re-reviewed. A resumed cycle re-runs the full regression suite rather than only the previously blocked cases.

13. Test Case Management & Traceability

Test cases are held in the repository alongside the code they verify, and each carries the requirement identifier it traces to. The RTM in the SRS appendix is the single authority for that mapping and is updated with execution status as testing proceeds. The suites are: Every functional, non-functional and security requirement in SRS v1.1 is represented in the suite mapping below.

Suite	Coverage	Requirements
TC-ACC-*	Account and access management	MLE-F-001 to F-003
TC-DS-*	Dataset upload, validation, profiling, ownership	MLE-F-010 to F-013
TC-PP-*	Preprocessing pipeline	MLE-F-020 to F-023
TC-MS-*	Model selection and hyperparameters	MLE-F-030 to F-032
TC-TR-*	Background training execution	MLE-F-040 to F-042
TC-EV-*	Metrics and run comparison	MLE-F-050 to F-052
TC-RP-*	Report generation and CSV export	MLE-F-060, F-061
TC-AD-*	Account and catalogue administration	MLE-F-062, F-063
TC-PERF-*	Response time and training duration	MLE-NF-001, NF-002
TC-REL-*	Failure isolation	MLE-NF-003
TC-UX-*	Usability and keyboard accessibility	MLE-NF-004
TC-MNT-*	Unit-test coverage of core modules	MLE-NF-005
TC-OPS-*	Retention and UTC timestamps	MLE-NF-006
TC-SEC-*	Security validation	MLE-SR-001 to SR-006

14. Test Metrics & Reporting

Metrics collected.

- Percentage of planned test cases executed.
- Percentage passed and failed.
- Requirement coverage — the proportion of requirements with at least one executed test case.
- Unit-test line coverage of PipelineApp, ModelsApp and EvaluationApp, against the 60% target of MLE-NF-005.
- Defect density by component, and defect aging for open defects.

Reports. An automated test report for every pull request, a weekly execution status summarised from Jira, and a final Test Summary Report stating which requirements passed, which defects remain open with their severity, and any test that could not be executed together with the reason.

15. Approvals

Role	Name	Signature / Date
ML Engineer / QA & Test	Disha Bose	
Data Pipeline Engineer	Chellamani Adithya Iyer	
Frontend & Results/Reports	R P Pranav	
Team Lead / Backend & Architecture	Denis Mangla	
Course Coordinator		